

Object-Oriented Programming in Java

Comprehensive Lecture Notes: Inheritance & Polymorphism

Course: Object-Oriented Programming

Topic: Inheritance, Subclasses, Method Overriding, Polymorphism & Dynamic Binding

Week: 03

Prerequisites: Week 02 (Classes, Objects, Static/Instance Members)

□ Table of Contents

1. [Learning Objectives](#)
 2. [Part 1: Introduction to Inheritance](#)
 3. [Part 2: Creating Subclasses](#)
 4. [Part 3: Method Overriding](#)
 5. [Part 4: The Substitution Principle](#)
 6. [Part 5: Dynamic Binding](#)
 7. [Part 6: Polymorphism in Action](#)
 8. [Summary & Quick Reference](#)
 9. [Practice Exercises](#)
 10. [Common Mistakes to Avoid](#)
-

Learning Objectives

By the end of this lecture, you should be able to:

- ✓ Explain the concept and benefits of inheritance
 - ✓ Create subclasses using the `extends` keyword
 - ✓ Understand constructor chaining with `super()`
 - ✓ Override inherited methods appropriately
 - ✓ Use the `@Override` annotation correctly
 - ✓ Apply the substitution principle (Liskov Substitution Principle)
 - ✓ Understand compile-time vs runtime method resolution
 - ✓ Explain and demonstrate dynamic binding
 - ✓ Implement polymorphic behavior using inheritance
 - ✓ Distinguish between method overriding and method overloading
-

Part 1: Introduction to Inheritance

1.1 What is Inheritance?

Definition: Inheritance is a mechanism where a new class (subclass) is created based on an existing class (superclass), inheriting its fields and methods.

Real-World Analogy:

- **Biology:** A child inherits traits from parents (eye color, height genes)
- **Vehicles:** A `Car` is a type of `Vehicle` (inherits wheels, engine, but adds specific features)
- **Geometry:** A `Cylinder` is an extended `Circle` (inherits radius, adds height)

The Problem: Code Duplication

Without inheritance, we often repeat code:

// ✖ WITHOUT INHERITANCE - Notice the duplication!

```
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() { return radius; }           // DUPLICATED
    public void setRadius(double r) { radius = r; }       // DUPLICATED

    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}

public class Cylinder {
    // ❌ DUPLICATE CODE from Circle!
    private double radius;           // DUPLICATED
    private double height;

    public Cylinder(double radius, double height) {
        this.radius = radius;        // DUPLICATED
        this.height = height;
    }

    public double getRadius() { return radius; }           // DUPLICATED
    public void setRadius(double r) { radius = r; }       // DUPLICATED

    public double getSurfaceArea() {
        return 2 * Math.PI * radius * (radius + height);
    }
}
```

```

    public double getVolume() {
        return Math.PI * radius * radius * height;
    }
}

```

Problems with this approach:

1. **Code Duplication** - radius and its methods appear twice
2. **Maintenance Nightmare** - Bug fix requires changes in multiple places
3. **Inconsistency Risk** - Easy to update one class but forget the other
4. **Violates DRY** - Don't Repeat Yourself principle

1.2 The Solution: Inheritance

// ✓ WITH INHERITANCE - No duplication!

```

public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() { return radius; }
    public void setRadius(double r) { radius = r; }

    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }
}

public class Cylinder extends Circle { // ✓ INHERITANCE!
    private double height;

    public Cylinder(double radius, double height) {
        super(radius); // Call parent constructor
        this.height = height;
    }

    public double getHeight() { return height; }

    // Cylinder automatically has:
    // - getRadius() (inherited)
    // - setRadius() (inherited)
    // - getArea() (inherited)
    // - getPerimeter() (inherited)

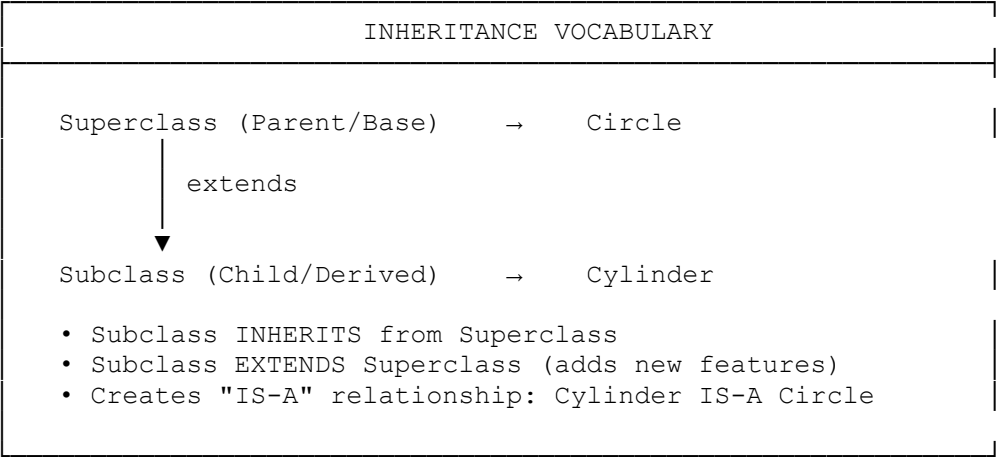
    // New methods specific to Cylinder:
    public double getVolume() {
        return getArea() * height; // Uses inherited getArea()!
    }
}

```

Benefits:

- ✓ **No code duplication** - radius exists only in Circle
- ✓ **Easier maintenance** - Fix bugs once in Circle
- ✓ **Consistency guaranteed** - All cylinders use same radius logic
- ✓ **Follows DRY principle**
- ✓ **Models real-world relationships** - "A Cylinder IS-A Circle (with height)"

1.3 Key Terminology



Terminology Table:

Term	Also Called	Example	Meaning
Superclass	Parent class, Base class	Circle	The class being inherited FROM
Subclass	Child class, Derived class	Cylinder	The class inheriting
extends	Inheritance keyword	class Cylinder extends Circle	Establishes inheritance
super	Parent reference	super(radius)	Refers to superclass members
IS-A	Relationship type	Cylinder IS-A Circle	Inheritance relationship

1.4 What Gets Inherited?

```
public class Circle {  
    // =====  
    // FIELDS  
    // =====  
    private double radius;           // ✓ Inherited (but not directly  
    accessible)  
    protected double circumference; // ✓ Inherited (accessible in  
    subclass)  
    public static final double PI = Math.PI; // ✓ Inherited (static  
    members too!)  
  
    // =====
```

Inheritance Rules:

❑ CRITICAL CONCEPT: Private Fields Are Inherited!

Memory Diagram:

Heap Memory - Single Object with TWO parts:



Cylinder Object	
Circle Part (Superclass):	
- radius: 5.0	(inherited)
+ getRadius()	(inherited)
+ getArea()	(inherited)
Cylinder Part (Subclass):	
- height: 10.0	(new field)
+ getVolume()	(new method)

1.5 Single Inheritance in Java

Java supports ONLY single inheritance (one parent class only).

```
// ✔ VALID - Single inheritance
public class Cylinder extends Circle { }

// ✘ INVALID - Multiple inheritance (Java doesn't allow this!)
public class Cylinder extends Circle, Rectangle { } // COMPILE ERROR!
```

Why only single inheritance?

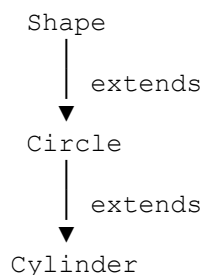
- Avoids the "Diamond Problem" (ambiguity when two parents have same method)
- Keeps class hierarchies simple and manageable
- Use **interfaces** for multiple type relationships (covered next week)

Inheritance Chain:

```
// You CAN have inheritance chains:

public class Shape { } // Grandparent
public class Circle extends Shape { } // Parent
public class Cylinder extends Circle { } // Child

// Cylinder inherits from BOTH Circle AND Shape
Inheritance Hierarchy:
```



Cylinder inherits from Circle, which inherits from Shape.
Therefore, Cylinder has members from BOTH Circle and Shape!

1.6 The IS-A Relationship

The Golden Rule: Inheritance should model an "IS-A" relationship.

```
// ✓ GOOD - Makes sense conceptually
class Cylinder extends Circle { }           // Cylinder IS-A Circle ✓
class Car extends Vehicle { }               // Car IS-A Vehicle ✓
class Student extends Person { }           // Student IS-A Person ✓
class Manager extends Employee { }         // Manager IS-A Employee ✓

// ✗ BAD - Doesn't make sense
class Car extends Garage { }               // Car IS-A Garage? ✗ NO!
class Student extends School { }           // Student IS-A School? ✗ NO!
class House extends Person { }             // House IS-A Person? ✗ NO!
```

IS-A vs HAS-A:

```
// IS-A → Use INHERITANCE
class Cylinder extends Circle { }
// "Cylinder IS-A Circle with height"

// HAS-A → Use COMPOSITION (instance variable)
class Car {
    private Engine engine; // Car HAS-A Engine
    private Wheel[] wheels; // Car HAS-AN array of Wheels
}
```

Decision Framework:

Ask yourself: "Is [Subclass] a type of [Superclass]?"

```
✓ "Is a Cylinder a type of Circle?" → YES → Use inheritance
✓ "Is a Car a type of Vehicle?" → YES → Use inheritance
✗ "Is a Car a type of Engine?" → NO → Use composition (Car HAS-A Engine)
✗ "Is a Student a type of School?" → NO → Use composition (School HAS-A Student)
```

Part 2: Creating Subclasses

2.1 Basic Subclass Syntax

```
// =====
// SUPERCLASS (Parent)
// =====
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() {
        return radius;
    }

    public void setRadius(double radius) {
        this.radius = radius;
    }
}
```

```

    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public String toString() {
        return String.format("Circle[radius=%.2f]", radius);
    }
}

// =====
// SUBCLASS (Child)
// =====
public class Cylinder extends Circle {
    // New field specific to Cylinder
    private double height;

    // Constructor
    public Cylinder(double radius, double height) {
        super(radius); // ⚠ MUST call parent constructor FIRST
        this.height = height;
    }

    // New methods specific to Cylinder
    public double getHeight() {
        return height;
    }

    public void setHeight(double height) {
        this.height = height;
    }

    public double getVolume() {
        // Uses inherited getArea() method!
        return getArea() * height;
    }

    @Override
    public String toString() {
        return String.format("Cylinder[radius=%.2f, height=%.2f]",
                               getRadius(), height);
    }
}

```

2.2 Constructor Chaining with super()

❑ **CRITICAL RULE:** The superclass constructor **MUST** be called in every subclass constructor.

```

public class Cylinder extends Circle {
    private double height;

    // ✓ EXPLICIT super() call
    public Cylinder(double radius, double height) {
        super(radius); // Calls Circle(double radius)
    }
}

```

```

        this.height = height;
    }

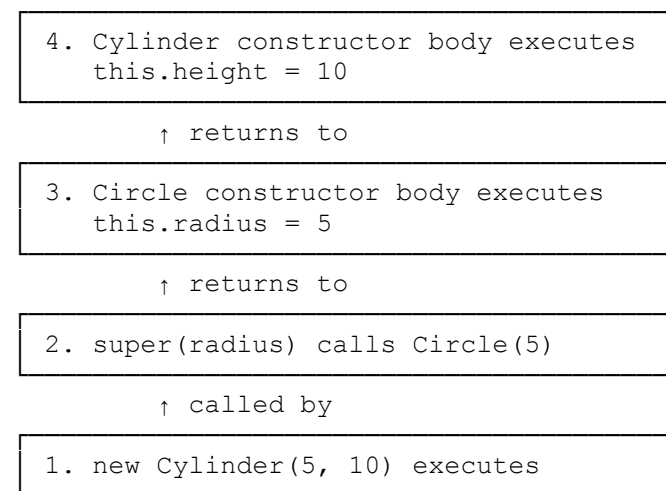
    // If you don't call super() explicitly, Java adds it automatically:
    public Cylinder(double radius, double height) {
        // super(); // ⚠ Java automatically adds super() with NO
arguments!
        this.height = height; // ✖ But Circle has no no-arg constructor!
    }
}

```

Constructor Chaining Visualization:

When you create: `Cylinder cy = new Cylinder(5, 10);`

Call Stack (bottom to top):



ORDER OF EXECUTION:

1. Subclass constructor called
2. super() called (parent constructor)
3. Parent constructor executes
4. Control returns to subclass constructor
5. Subclass constructor completes

super() Rules:

super() RULES
1. Must be FIRST statement in constructor (if present) 2. Can only call ONE parent constructor 3. If not present, Java adds super() automatically 4. Parent class must have matching constructor 5. Cannot use super() and this() in same constructor

Examples:

```

public class Circle {
    private double radius;

    // No-arg constructor
}

```

```

    public Circle() {
        this.radius = 1.0;
    }

    // Parameterized constructor
    public Circle(double radius) {
        this.radius = radius;
    }
}

public class Cylinder extends Circle {
    private double height;

    // ✓ Example 1: Call specific parent constructor
    public Cylinder(double radius, double height) {
        super(radius); // Calls Circle(double)
        this.height = height;
    }

    // ✓ Example 2: Call no-arg parent constructor explicitly
    public Cylinder(double height) {
        super(); // Calls Circle() - radius will be 1.0
        this.height = height;
    }

    // ✓ Example 3: Implicit super() call
    public Cylinder() {
        // super(); // Java adds this automatically
        this.height = 1.0;
    }

    // ✗ Example 4: COMPILE ERROR - super() not first
    public Cylinder(double radius, double height) {
        this.height = height; // ✗ Must call super() FIRST!
        super(radius);        // ✗ Too late!
    }

    // ✗ Example 5: COMPILE ERROR - both super() and this()
    public Cylinder(double radius, double height) {
        this(height); // Calls another constructor
        super(radius); // ✗ Can't have both!
    }
}

```

2.3 Complete Example: Rectangle and Cuboid

```

// =====
// SUPERCLASS: Rectangle
// =====
package com.iit.shapes;

public class Rectangle {
    // Instance variables
    private double width;
    private double height;

    // No-arg constructor
    public Rectangle() {
        this(1.0, 1.0); // Constructor chaining
    }
}

```

```

    }

    // Parameterized constructor
    public Rectangle(double width, double height) {
        if (width <= 0 || height <= 0) {
            throw new IllegalArgumentException("Dimensions must be
positive");
        }
        this.width = width;
        this.height = height;
    }

    // Getters
    public double getWidth() {
        return width;
    }

    public double getHeight() {
        return height;
    }

    // Setters
    public void setWidth(double width) {
        if (width <= 0) {
            throw new IllegalArgumentException("Width must be positive");
        }
        this.width = width;
    }

    public void setHeight(double height) {
        if (height <= 0) {
            throw new IllegalArgumentException("Height must be positive");
        }
        this.height = height;
    }

    // Calculate area
    public double getArea() {
        return width * height;
    }

    // Calculate perimeter
    public double getPerimeter() {
        return 2 * (width + height);
    }

    @Override
    public String toString() {
        return String.format("Rectangle[width=%.2f, height=%.2f,
area=%.2f]",
                                width, height, getArea());
    }
}

// =====
// SUBCLASS: Cuboid (3D Rectangle)
// =====
package com.iit.shapes;

public class Cuboid extends Rectangle {
    // New field for third dimension

```

```

private double depth;

// No-arg constructor
public Cuboid() {
    super();          // Calls Rectangle() → width=1, height=1
    this.depth = 1.0;
}

// Parameterized constructor
public Cuboid(double width, double height, double depth) {
    super(width, height); // Initialize inherited dimensions
    if (depth <= 0) {
        throw new IllegalArgumentException("Depth must be positive");
    }
    this.depth = depth;
}

// Getter for depth
public double getDepth() {
    return depth;
}

// Setter for depth
public void setDepth(double depth) {
    if (depth <= 0) {
        throw new IllegalArgumentException("Depth must be positive");
    }
    this.depth = depth;
}

// Calculate volume (new method, specific to Cuboid)
public double getVolume() {
    // Uses inherited getArea() for base area!
    return getArea() * depth;
    // OR: return getWidth() * getHeight() * depth;
}

// Get surface area of base (inherited method still works)
public double getBaseArea() {
    return super.getArea(); // Explicitly call parent's getArea()
}

@Override
public String toString() {
    return String.format("Cuboid[width=%.2f, height=%.2f, depth=%.2f, volume=%.2f]",
        getWidth(), getHeight(), depth, getVolume());
}
}

```

Using the Classes:

```

public class ShapeDemo {
    public static void main(String[] args) {
        // =====
        // Creating Rectangle objects
        // =====
        Rectangle rect1 = new Rectangle();           // 1x1 rectangle
        Rectangle rect2 = new Rectangle(4, 3);       // 4x3 rectangle
    }
}

```

```

        System.out.println(rect1); // Rectangle[width=1.00, height=1.00,
area=1.00]
        System.out.println(rect2); // Rectangle[width=4.00, height=3.00,
area=12.00]

        System.out.println("Perimeter: " + rect2.getPerimeter()); // 14.0

        // =====
        // Creating Cuboid objects
        // =====
        Cuboid cuboid1 = new Cuboid(); // 1x1x1 cube
        Cuboid cuboid2 = new Cuboid(4, 3, 2); // 4x3x2 cuboid

        System.out.println(cuboid1); // Cuboid[width=1.00, height=1.00,
depth=1.00, volume=1.00]
        System.out.println(cuboid2); // Cuboid[width=4.00, height=3.00,
depth=2.00, volume=24.00]

        // =====
        // Using inherited methods
        // =====
        System.out.println("Width: " + cuboid2.getWidth()); // 4.0
(inherited)
        System.out.println("Height: " + cuboid2.getHeight()); // 3.0
(inherited)
        System.out.println("Base area: " + cuboid2.getBaseArea()); // 12.0
(inherited)

        // =====
        // Using new methods
        // =====
        System.out.println("Depth: " + cuboid2.getDepth()); // 2.0
(new)
        System.out.println("Volume: " + cuboid2.getVolume()); // 24.0
(new)

        // =====
        // Modifying using inherited setters
        // =====
        cuboid2.setWidth(5); // Uses inherited setter
        cuboid2.setDepth(3); // Uses own setter
        System.out.println("New volume: " + cuboid2.getVolume()); // 45.0
    }
}

```

Output:

```

Rectangle[width=1.00, height=1.00, area=1.00]
Rectangle[width=4.00, height=3.00, area=12.00]
Perimeter: 14.0
Cuboid[width=1.00, height=1.00, depth=1.00, volume=1.00]
Cuboid[width=4.00, height=3.00, depth=2.00, volume=24.00]
Width: 4.0
Height: 3.0
Base area: 12.0
Depth: 2.0
Volume: 24.0
New volume: 45.0

```

2.4 Accessing Inherited Members

```
public class Cylinder extends Circle {
    private double height;

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public void demonstrateAccess() {
        // =====
        // ACCESSING INHERITED MEMBERS
        // =====

        // ✓ Access public inherited methods
        double r = getRadius();    // Calls inherited getter
        setRadius(7.5);            // Calls inherited setter

        // ✓ Access inherited methods implicitly
        double area = getArea();    // Calls inherited method

        // ✗ Cannot access private inherited fields directly
        // System.out.println(radius); // COMPILER ERROR!

        // ✓ Must use inherited getter instead
        System.out.println(getRadius()); // OK

        // =====
        // USING super KEYWORD
        // =====

        // Access parent's method explicitly (useful when overridden)
        double circleArea = super.getArea();

        // Access own field
        double h = this.height;

        // Access inherited method
        double r2 = this.getRadius(); // Same as getRadius()
    }
}
```

Part 3: Method Overriding

3.1 What is Method Overriding?

Definition: Method overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass.

Why Override?

- Customize inherited behavior for the subclass
- Implement polymorphism (different behavior for different types)
- Specialize general methods

Example: Circle vs Cylinder Area

```
public class Circle {
    private double radius;

    // getArea() for Circle returns area of 2D circle
    public double getArea() {
        return Math.PI * radius * radius; //  $\pi r^2$ 
    }
}

public class Cylinder extends Circle {
    private double height;

    // Override getArea() for Cylinder to return surface area
    @Override
    public double getArea() {
        // Surface area of cylinder =  $2\pi r(r + h)$ 
        double r = getRadius();
        return 2 * Math.PI * r * (r + height);
    }
}
```

Why is this useful?

- Circle's `getArea()` returns 2D area (πr^2)
- Cylinder's `getArea()` returns 3D surface area ($2\pi r(r+h)$)
- Same method name, different behavior based on object type!

3.2 The @Override Annotation

Purpose: Tells the compiler "I intend to override a parent method."

```
public class Cylinder extends Circle {

    // ✓ WITH @Override - Compiler checks this is actually overriding
    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * (getRadius() + height);
    }
}
```

```

    // ✗ Typo in method name - @Override catches this!
    @Override
    public double getAraa() { // ✗ COMPILE ERROR: method does not
override
        return 0;
    }
}

```

Benefits of @Override:

1. **Catches typos** - Compiler error if method doesn't actually override
2. **Documents intent** - Clear that you meant to override
3. **Future-proof** - If parent method changes, you'll know
4. **Best practice** - Always use it when overriding!

```

// ✗ WITHOUT @Override - Typo creates NEW method (not override)
public class Cylinder extends Circle {
    public double getAraa() { // Typo! Created new method, didn't override
        return 0;
    }
}

```

```

// ✔ WITH @Override - Compiler catches the mistake
public class Cylinder extends Circle {
    @Override
    public double getAraa() { // ✗ COMPILE ERROR: no such method in
parent
        return 0;
    }
}

```

3.3 Method Overriding Rules

For a method to properly override another:

METHOD OVERRIDING REQUIREMENTS
1. SAME METHOD NAME 2. SAME PARAMETER LIST (type, number, order) 3. SAME OR COMPATIBLE RETURN TYPE (covariant return types OK) 4. SAME OR MORE ACCESSIBLE (can't reduce visibility) 5. CANNOT override final, static, or private methods 6. CAN change throws clause (fewer exceptions)

Detailed Examples:

```

public class Circle {
    // Original method
    public double getArea() {
        return Math.PI * radius * radius;
    }

    protected void display() {
        System.out.println("Circle");
    }
}

```

```

}

public class Cylinder extends Circle {

    // ✓ VALID - Same name, parameters, compatible return type
    @Override
    public double getArea() {
        return 2 * Math.PI * getRadius() * (getRadius() + height);
    }

    // ✓ VALID - Same or MORE accessible (protected → public)
    @Override
    public void display() {
        System.out.println("Cylinder");
    }

    // ✗ INVALID - Different parameter list (this is OVERLOADING, not
    overriding)
    @Override // COMPILE ERROR!
    public double getArea(boolean includeBase) {
        return 0;
    }

    // ✗ INVALID - Different return type
    @Override // COMPILE ERROR!
    public int getArea() {
        return 0;
    }

    // ✗ INVALID - Less accessible (public → private)
    @Override // COMPILE ERROR!
    private void display() {
        System.out.println("Cylinder");
    }
}

```

3.4 Overriding vs Overloading

❗ **CRITICAL: Don't confuse overriding with overloading!**

		OVERRIDING		OVERLOADING
Relationship		Subclass ↔ Superclass		Same class
Method name		SAME		SAME
Parameters		SAME		DIFFERENT
Return type		SAME (or covariant)		Can be different
Access modifier		Same or more accessible		Any
When resolved (static)		RUNTIME (dynamic binding)		COMPILE TIME

Keyword	@Override	None
Purpose	Change behavior	Convenience

Side-by-Side Example:

```

public class Circle {
    // Original method
    public double getArea() {
        return Math.PI * radius * radius;
    }
}

public class Cylinder extends Circle {

    // =====
    // METHOD OVERRIDING - Same signature as parent
    // =====
    @Override
    public double getArea() { // SAME name, SAME parameters
        return 2 * Math.PI * getRadius() * (getRadius() + height);
    }

    // =====
    // METHOD OVERLOADING - Different parameters
    // =====
    public double getArea(boolean includeBases) { // SAME name, DIFFERENT
parameters
        if (includeBases) {
            return 2 * Math.PI * getRadius() * (getRadius() + height);
        } else {
            return 2 * Math.PI * getRadius() * height; // Lateral area
only
        }
    }
}

// Usage
Circle c = new Circle(5);
Cylinder cy = new Cylinder(5, 10);

c.getArea(); // Calls Circle's version →  $\pi r^2$ 
cy.getArea(); // Calls Cylinder's OVERRIDDEN version →  $2\pi r(r+h)$ 
cy.getArea(true); // Calls OVERLOADED version

```

3.5 Using super to Access Parent Methods

```

public class Cylinder extends Circle {
    private double height;

    // Override getArea() to return surface area
    @Override
    public double getArea() {
        // Cylinder surface area:  $2\pi r(r + h)$ 
        double r = getRadius();
        return 2 * Math.PI * r * (r + height);
    }
}

```

```

    }

    // New method: get area of circular base
    public double getBaseArea() {
        // ✓ Use super to call parent's getArea()
        return super.getArea(); // Calls Circle's getArea() →  $\pi r^2$ 
    }

    // Calculate volume using parent's method
    public double getVolume() {
        // Use parent's getArea() for base area
        return super.getArea() * height;
        // OR use getBaseArea(): return getBaseArea() * height;
    }

    @Override
    public String toString() {
        // ✓ Call parent's toString() first, then add own info
        return super.toString() + ", height=" + height;
    }
}

```

Example Usage:

```
Cylinder cy = new Cylinder(5, 10);
```

```

cy.getArea();           // 471.24 (surface area - overridden version)
cy.getBaseArea();       // 78.54 (base area - uses super.getArea())
cy.getVolume();         // 785.40 (uses super.getArea() for base)

```

super vs this:

Keyword	Purpose
this	Refers to CURRENT object - this.field → current object's field - this.method() → current object's method - this() → call another constructor in SAME class
super	Refers to PARENT class - super.field → parent's field (if accessible) - super.method() → parent's version of method - super() → call parent's constructor

3.6 Complete Overriding Example

```

// =====
// PARENT CLASS
// =====
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    public double getRadius() {

```

```

        return radius;
    }

    // Method to be overridden
    public double getArea() {
        return Math.PI * radius * radius;
    }

    public double getPerimeter() {
        return 2 * Math.PI * radius;
    }

    @Override
    public String toString() {
        return String.format("Circle[radius=%.2f]", radius);
    }
}

// =====
// CHILD CLASS WITH OVERRIDING
// =====
public class Cylinder extends Circle {
    private double height;

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    public double getHeight() {
        return height;
    }

    // =====
    // OVERRIDDEN METHOD - Different behavior than parent
    // =====
    @Override
    public double getArea() {
        // Total surface area of cylinder
        double r = getRadius();
        return 2 * Math.PI * r * (r + height);
    }

    // =====
    // NEW METHODS - Not in parent
    // =====
    public double getVolume() {
        // Use parent's getArea() for base
        return super.getArea() * height;
    }

    public double getLateralArea() {
        // Just the curved surface
        return 2 * Math.PI * getRadius() * height;
    }

    public double getBaseArea() {
        // Explicitly call parent's getArea()
        return super.getArea();
    }
}

```

```

        @Override
        public String toString() {
            return String.format("Cylinder[radius=%.2f, height=%.2f]",
                                getRadius(), height);
        }
    }

// =====
// DEMONSTRATION
// =====
public class OverridingDemo {
    public static void main(String[] args) {
        Circle circle = new Circle(5);
        Cylinder cylinder = new Cylinder(5, 10);

        System.out.println("=== Circle ===");
        System.out.println(circle);
        System.out.println("Area: " + circle.getArea());           // 78.54
        (πr²) System.out.println("Perimeter: " + circle.getPerimeter()); // 31.42

        System.out.println("\n=== Cylinder ===");
        System.out.println(cylinder);
        System.out.println("Surface Area: " + cylinder.getArea()); //
471.24 (OVERRIDDEN!)
        System.out.println("Base Area: " + cylinder.getBaseArea()); //
78.54 (uses super)
        System.out.println("Lateral Area: " +
cylinder.getLateralArea()); // 314.16
        System.out.println("Volume: " + cylinder.getVolume());     //
785.40
        System.out.println("Perimeter: " + cylinder.getPerimeter()); //
31.42 (inherited)
    }
}

```

Output:

```

=== Circle ===
Circle[radius=5.00]
Area: 78.53981633974483
Perimeter: 31.41592653589793

=== Cylinder ===
Cylinder[radius=5.00, height=10.00]
Surface Area: 471.23889803846896
Base Area: 78.53981633974483
Lateral Area: 314.1592653589793
Volume: 785.3981633974483
Perimeter: 31.41592653589793

```

Part 4: The Substitution Principle

4.1 What is the Substitution Principle?

Formal Definition (Liskov Substitution Principle):

"Objects of a superclass should be replaceable with objects of a subclass without breaking the application."

Simple Translation:

"Anywhere you can use a parent class object, you should be able to use a child class object instead."

Why It Matters:

- Foundation of polymorphism
- Enables flexible, reusable code
- Models real-world "IS-A" relationships

4.2 Basic Substitution

```
// ✓ NORMAL: Each variable holds its exact type
Circle c1 = new Circle(5);
Cylinder cyl = new Cylinder(5, 10);

// ✓ SUBSTITUTION: Superclass reference holds subclass object
Circle c2 = new Cylinder(3, 7); // LEGAL! Cylinder IS-A Circle
```

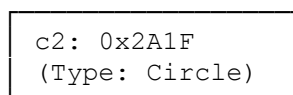
Why is this legal?

- Cylinder extends Circle
- Cylinder IS-A Circle (with additional height)
- Every Cylinder has everything a Circle has
- Therefore, a Cylinder can be used wherever a Circle is expected

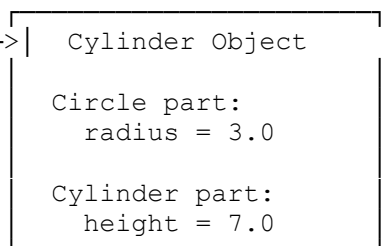
Memory Diagram:

```
Circle c2 = new Cylinder(3, 7);
```

Stack:



Heap:



```
c2's COMPILE-TIME type: Circle
c2's RUNTIME type: Cylinder
```

4.3 What Can You Access?

❑ **CRITICAL RULE:** The reference type determines what members you can ACCESS (compile-time check).

```
Circle c = new Cylinder(5, 10);

// ✔ Can call Circle methods (c's type is Circle)
c.getRadius();      // ✔ Circle has getRadius()
c.getArea();        // ✔ Circle has getArea()
c.getPerimeter();   // ✔ Circle has getPerimeter()

// ✗ Cannot call Cylinder-specific methods (c's type is Circle)
c.getHeight();      // ✗ COMPILE ERROR: Circle doesn't have getHeight()
c.getVolume();      // ✗ COMPILE ERROR: Circle doesn't have getVolume()

// BUT the ACTUAL object is still a Cylinder!
// So when you call getArea(), it runs Cylinder's overridden version!
```

Access Rules:

WHAT YOU CAN ACCESS WITH SUBSTITUTION
Circle c = new Cylinder(5, 10);
✔ All methods declared in Circle - getRadius(), setRadius() - getArea(), getPerimeter()
✗ Methods ONLY in Cylinder (not in Circle) - getHeight(), setHeight() - getVolume(), getLateralArea()
⚠ If method is OVERRIDDEN: → Cylinder's version runs (runtime decision)

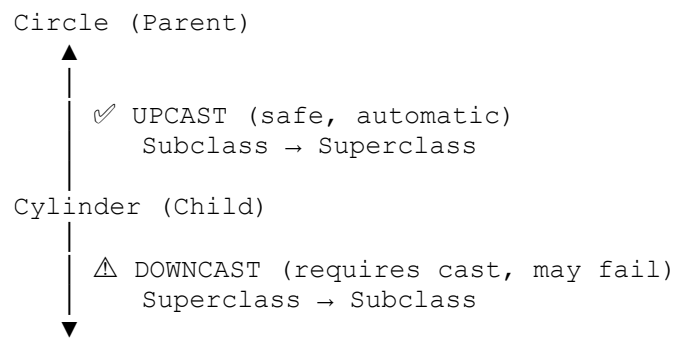
4.4 The Direction Rule

```
// ✔ UPCAST (Subclass → Superclass) - ALWAYS legal, automatic
Cylinder cy = new Cylinder(5, 10);
Circle c = cy; // Implicit upcast (no cast needed)

// ✗ DOWNCAST (Superclass → Subclass) - Needs explicit cast, may fail
Circle c = new Circle(5);
Cylinder cy = c; // ✗ COMPILE ERROR: incompatible types

Cylinder cy = (Cylinder) c; // Compiles, but RUNTIME ERROR!
// ClassCastException: Circle cannot be cast to Cylinder
```

Visual:



Upcast vs Downcast Examples:

```
// =====
// UPCAST - Safe, automatic
// =====
Cylinder cy = new Cylinder(5, 10);

Circle c1 = cy;                // ✓ Implicit upcast
Circle c2 = (Circle) cy;       // ✓ Explicit (unnecessary)

// Both work because Cylinder IS-A Circle

// =====
// DOWNCAST - Risky, requires explicit cast
// =====

// Case 1: Object IS actually a Cylinder
Circle c = new Cylinder(5, 10);
Cylinder cy1 = (Cylinder) c;    // ✓ OK - object really is Cylinder

// Case 2: Object is just a Circle
Circle c2 = new Circle(5);
Cylinder cy2 = (Cylinder) c2;    // ✗ RUNTIME ERROR: ClassCastException!

// =====
// SAFE DOWNCAST - Check type first
// =====
Circle c3 = new Cylinder(5, 10);

if (c3 instanceof Cylinder) {    // Check before casting
    Cylinder cy3 = (Cylinder) c3;
    System.out.println(cy3.getVolume()); // ✓ Safe!
}
```

4.5 Practical Use: Polymorphic Parameters

Example 1: Method that accepts any Circle

```
public class ShapeCalculator {

    // Method accepts Circle OR any subclass of Circle
    public static void printCircleInfo(Circle c) {
        System.out.println("Radius: " + c.getRadius());
        System.out.println("Area: " + c.getArea());
    }
}
```

```

        System.out.println("Perimeter: " + c.getPerimeter());
        System.out.println();
    }

    public static void main(String[] args) {
        Circle circle = new Circle(5);
        Cylinder cylinder = new Cylinder(5, 10);

        // ✓ Both work - substitution principle!
        printCircleInfo(circle);    // Pass Circle
        printCircleInfo(cylinder);  // Pass Cylinder (upcast happens)
    }
}

```

Output:

```

Radius: 5.0
Area: 78.53981633974483      ← Circle's getArea()
Perimeter: 31.41592653589793

Radius: 5.0
Area: 471.23889803846896    ← Cylinder's OVERRIDDEN getArea()!
Perimeter: 31.41592653589793

```

Notice: Even though parameter type is `Circle`, when we pass a `Cylinder`, it calls `Cylinder`'s overridden `getArea()` method! This is **dynamic binding** (covered next).

Example 2: Return Type Substitution

```

public class ShapeFactory {

    // Returns Circle, but could return Cylinder
    public static Circle createShape(String type, double radius, double
height) {
        if (type.equals("cylinder")) {
            return new Cylinder(radius, height); // ✓ Return Cylinder as
Circle
        } else {
            return new Circle(radius);
        }
    }

    public static void main(String[] args) {
        Circle c1 = createShape("circle", 5, 0);
        Circle c2 = createShape("cylinder", 5, 10); // Actually a
Cylinder!

        System.out.println(c1.getArea()); // 78.54
        System.out.println(c2.getArea()); // 471.24 (Cylinder's version!)
    }
}

```

4.6 Array Substitution

One of the most powerful uses of substitution:

```
// ✓ Array of superclass can hold subclass objects
Circle[] shapes = new Circle[4];

shapes[0] = new Circle(3);           // Circle object
shapes[1] = new Cylinder(2, 5);      // Cylinder object (upcast)
shapes[2] = new Circle(4);           // Circle object
shapes[3] = new Cylinder(3, 8);      // Cylinder object (upcast)

// Loop through heterogeneous array
for (Circle shape : shapes) {
    System.out.println(shape.getClass().getSimpleName() +
                        " area: " + shape.getArea());
}
```

Output:

```
Circle area: 28.274333882308138
Cylinder area: 87.96459430051421    ← Called Cylinder's version!
Circle area: 50.26548245743669
Cylinder area: 207.34511513692632   ← Called Cylinder's version!
```

This is POLYMORPHISM in action! Same method call (`getArea()`), different behavior based on actual object type.

Part 5: Dynamic Binding

5.1 What is Dynamic Binding?

Definition: The process of determining which method implementation to call at **RUNTIME** based on the **actual object type**, not the reference type.

Also called:

- Late binding
- Runtime polymorphism
- Virtual method invocation

Contrast with Static Binding:

	STATIC BINDING	DYNAMIC BINDING
When resolved	COMPILE TIME	RUNTIME
Based on	Reference type	Actual object type
Applies to	static methods, private methods, final methods	Instance methods (overridden methods)
Also called	Early binding	Late binding
Can be overridden	NO	YES

5.2 How Dynamic Binding Works

```
Circle c1 = new Circle(5);  
Circle c2 = new Cylinder(5, 10); // Reference: Circle, Object: Cylinder  
  
c1.getArea(); // Which getArea() is called?  
c2.getArea(); // Which getArea() is called?
```

Step-by-Step Process:

COMPILE TIME:

1. Compiler sees: `c1.getArea()` and `c2.getArea()`
2. Compiler checks: Does Circle class have `getArea()`? ✓ YES
3. Compiler allows both calls
4. Compiler DOES NOT decide which implementation yet

RUNTIME:

1. JVM looks at `c1`'s actual object → Circle object
 2. JVM calls Circle's `getArea()` → returns πr^2
-
1. JVM looks at `c2`'s actual object → Cylinder object
 2. JVM sees Cylinder has overridden `getArea()`
 3. JVM calls Cylinder's `getArea()` → returns $2\pi r(r+h)$

Visual Timeline:

```
Code: Circle c = new Cylinder(5, 10);  
      c.getArea();
```

COMPILE TIME (javac compiler):

1. Check: `c`'s type is Circle
2. Question: Does Circle have `getArea()`?
3. Answer: YES ✓
4. Compiler: "This call is valid"
5. Compiler: "I'll decide LATER which one"

RUNTIME (JVM):

1. Look at actual object `c` points to
2. Object type: Cylinder
3. Question: Did Cylinder override `getArea()`?
4. Answer: YES ✓

<pre>5. JVM: "Call Cylinder's getArea()" 6. Execute: return 2πr(r+h)</pre>
--

5.3 Detailed Example

```
public class DynamicBindingDemo {
    public static void main(String[] args) {

        // =====
        // Setup: Same reference type, different object types
        // =====
        Circle c1 = new Circle(5);
        Circle c2 = new Cylinder(5, 10);

        System.out.println("=== Method Calls ===");

        // =====
        // DYNAMIC BINDING - Resolved at runtime
        // =====
        System.out.println("c1.getArea(): " + c1.getArea());
        // → JVM sees: c1 points to Circle object
        // → Calls: Circle's getArea()
        // → Result: 78.54 (πr2)

        System.out.println("c2.getArea(): " + c2.getArea());
        // → JVM sees: c2 points to Cylinder object
        // → Cylinder overrode getArea()
        // → Calls: Cylinder's getArea()
        // → Result: 471.24 (2πr(r+h))

        // =====
        // Both calls look identical in code!
        // But runtime behavior is DIFFERENT based on actual object
        // =====

        System.out.println("\n=== Object Types ===");
        System.out.println("c1 actual type: " + c1.getClass().getName());
        System.out.println("c2 actual type: " + c2.getClass().getName());
    }
}
```

Output:

```
=== Method Calls ===
c1.getArea(): 78.53981633974483
c2.getArea(): 471.23889803846896
```

```
=== Object Types ===
c1 actual type: Circle
c2 actual type: Cylinder
```

5.4 Static vs Dynamic Binding Example

```
public class Circle {
    private double radius;

    public Circle(double radius) {
        this.radius = radius;
    }
}
```

```

    }

    // =====
    // INSTANCE METHOD - Uses dynamic binding
    // =====
    public double getArea() {
        System.out.println("Circle's getArea()");
        return Math.PI * radius * radius;
    }

    // =====
    // STATIC METHOD - Uses static binding
    // =====
    public static void printInfo() {
        System.out.println("This is a Circle");
    }
}

public class Cylinder extends Circle {
    private double height;

    public Cylinder(double radius, double height) {
        super(radius);
        this.height = height;
    }

    // =====
    // OVERRIDE instance method - Dynamic binding
    // =====
    @Override
    public double getArea() {
        System.out.println("Cylinder's getArea()");
        return 2 * Math.PI * getRadius() * (getRadius() + height);
    }

    // =====
    // "OVERRIDE" static method - NOT actually overriding!
    // This HIDES the parent's static method
    // =====
    public static void printInfo() {
        System.out.println("This is a Cylinder");
    }
}

public class BindingDemo {
    public static void main(String[] args) {
        Circle c = new Cylinder(5, 10);

        // =====
        // DYNAMIC BINDING (instance method)
        // =====
        c.getArea(); // Calls Cylinder's version (based on object type)
        // Output: "Cylinder's getArea()"

        // =====
        // STATIC BINDING (static method)
        // =====
        c.printInfo(); // Calls Circle's version (based on reference
type!)
        // Output: "This is a Circle"

```

```

        // ⚠ Notice: printInfo() called Circle's version even though
        //     the object is actually a Cylinder!
        //     This is because static methods use STATIC BINDING
    }
}

```

Output:

```

Cylinder's getArea()
This is a Circle

```

Key Takeaway:

- **Instance methods** → Dynamic binding (runtime, based on object)
- **Static methods** → Static binding (compile time, based on reference)

Part 6: Polymorphism in Action

6.1 What is Polymorphism?

Etymology: Greek - "poly" (many) + "morph" (forms) = many forms

Definition: The ability of a single interface to represent different underlying forms (data types).

In OOP: Same method name behaves differently based on the object it's called on.

Three Types of Polymorphism in Java:

1. **Compile-time polymorphism** - Method overloading (covered Week 02)
2. **Runtime polymorphism** - Method overriding (this week)
3. **Parametric polymorphism** - Generics (future topic)

We focus on Runtime Polymorphism (Inheritance-based).

6.2 The Power of Polymorphism

Example: Processing Different Shapes

```

public class PolymorphismDemo {

    // =====
    // ONE METHOD processes ANY Circle (including subclasses)
    // =====
    public static void analyzeCircle(Circle c) {
        System.out.println("Object type: " + c.getClass().getSimpleName());
        System.out.println("Radius: " + c.getRadius());
        System.out.println("Area: " + c.getArea()); // Polymorphic!
        System.out.println("Perimeter: " + c.getPerimeter());
    }
}

```

```

        System.out.println();
    }

    public static void main(String[] args) {
        Circle circle = new Circle(5);
        Cylinder cylinder = new Cylinder(5, 10);

        // ✔ Same method, different objects, different behavior!
        analyzeCircle(circle);
        analyzeCircle(cylinder); // Automatic upcast
    }
}

```

Output:

```

Object type: Circle
Radius: 5.0
Area: 78.53981633974483
Perimeter: 31.41592653589793

Object type: Cylinder
Radius: 5.0
Area: 471.23889803846896 ← Different behavior! Polymorphism!
Perimeter: 31.41592653589793

```

Without Polymorphism, you'd need:

```

// ✘ WITHOUT POLYMORPHISM - Need separate methods
public static void analyzeCircle(Circle c) { ... }
public static void analyzeCylinder(Cylinder cy) { ... }

// ✘ AND separate calls
analyzeCircle(circle);
analyzeCylinder(cylinder);

```

6.3 Heterogeneous Collections

The most powerful use of polymorphism: storing different types in one array!

```

public class PolymorphicArray {
    public static void main(String[] args) {

        // =====
        // Create heterogeneous array - mix of Circle and Cylinder
        // =====
        Circle[] shapes = {
            new Circle(3),
            new Cylinder(2, 5),
            new Circle(4),
            new Cylinder(3, 8),
            new Cylinder(1, 10),
            new Circle(6)
        };

        // =====
        // Process all shapes uniformly with polymorphic behavior
        // =====
        System.out.println("=== All Shapes ===");
    }
}

```

```

    for (Circle shape : shapes) {
        System.out.printf("%s (r=%.1f): Area = %.2f%n",
            shape.getClass().getSimpleName(),
            shape.getRadius(),
            shape.getArea()); // ✨ POLYMORPHIC CALL!
    }

    // =====
    // Calculate total area polymorphically
    // =====
    double totalArea = 0;
    for (Circle shape : shapes) {
        totalArea += shape.getArea(); // Each calls its own version!
    }
    System.out.printf("%nTotal area: %.2f%n", totalArea);

    // =====
    // Count by type
    // =====
    int circleCount = 0, cylinderCount = 0;
    for (Circle shape : shapes) {
        if (shape instanceof Cylinder) {
            cylinderCount++;
        } else {
            circleCount++;
        }
    }
    System.out.println("Circles: " + circleCount + ", Cylinders: " +
        cylinderCount);
}
}

```

Output:

```

=== All Shapes ===
Circle (r=3.0): Area = 28.27
Cylinder (r=2.0): Area = 87.96
Circle (r=4.0): Area = 50.27
Cylinder (r=3.0): Area = 207.35
Cylinder (r=1.0): Area = 69.12
Circle (r=6.0): Area = 113.10

```

```

Total area: 556.07
Circles: 3, Cylinders: 3

```

6.4 Polymorphism with Rectangle/Cuboid

```

public class RectanglePolymorphism {
    public static void main(String[] args) {

        // =====
        // Heterogeneous array of rectangles and cuboids
        // =====
        Rectangle[] boxes = {
            new Rectangle(2, 3),
            new Cuboid(2, 3, 4),
            new Rectangle(5, 5),
            new Cuboid(1, 1, 1),
            new Cuboid(3, 4, 5),

```

```

        new Rectangle(10, 2)
    };

    System.out.println("=== Processing Rectangles ===");
    for (Rectangle box : boxes) {
        String type = box.getClass().getSimpleName();
        double area = box.getArea(); // Polymorphic!

        System.out.printf("%s: Area = %.2f", type, area);

        // Type-specific processing
        if (box instanceof Cuboid) {
            Cuboid cuboid = (Cuboid) box; // Safe downcast
            System.out.printf(", Volume = %.2f", cuboid.getVolume());
        }
        System.out.println();
    }

    // =====
    // Calculate statistics polymorphically
    // =====
    double totalArea = 0;
    double totalVolume = 0;

    for (Rectangle box : boxes) {
        totalArea += box.getArea();

        if (box instanceof Cuboid) {
            totalVolume += ((Cuboid) box).getVolume();
        }
    }

    System.out.printf("\nTotal surface area: %.2f\n", totalArea);
    System.out.printf("Total volume: %.2f\n", totalVolume);
}
}

```

6.5 Benefits of Polymorphism

BENEFITS OF POLYMORPHISM	
1. CODE REUSABILITY	- Write one method that works with multiple types - Don't duplicate code for Circle vs Cylinder
2. EXTENSIBILITY	- Add new subclasses without changing existing code - Example: Add Cone class, existing methods still work
3. MAINTAINABILITY	- Fix bugs once in superclass - Changes automatically propagate to subclasses
4. FLEXIBILITY	- Work with heterogeneous collections - Process different types uniformly
5. ABSTRACTION	- Work with general concepts (Circle)

```
- Ignore specific details (whether it's Cylinder or not)
```

6.6 Real-World Polymorphism Example

```
/**
 * Real-world example: Shape drawing application
 */
public class DrawingApp {

    // =====
    // Polymorphic method - works with ANY Circle
    // =====
    public static double calculateMaterialCost(Circle shape, double
costPerUnit) {
        // getArea() is polymorphic - calls appropriate version
        return shape.getArea() * costPerUnit;
    }

    public static void printShapeReport(Circle shape) {
        System.out.println("=====");
        System.out.println("    " + shape.getClass().getSimpleName() + "
Report");
        System.out.println("=====");
        System.out.println("    Radius: " + shape.getRadius());
        System.out.println("    Area: " + String.format("%.2f",
shape.getArea()));
        System.out.println("    Perimeter: " + String.format("%.2f",
shape.getPerimeter()));

        // Type-specific information
        if (shape instanceof Cylinder) {
            Cylinder cy = (Cylinder) shape;
            System.out.println("    Height: " + cy.getHeight());
            System.out.println("    Volume: " + String.format("%.2f",
cy.getVolume()));
        }

        System.out.println("=====");
    }

    public static void main(String[] args) {
        Circle circle = new Circle(5);
        Cylinder cylinder = new Cylinder(5, 10);

        // =====
        // Polymorphic method calls
        // =====
        printShapeReport(circle);
        System.out.println("Material cost: $" +
calculateMaterialCost(circle, 2.50));
        System.out.println();

        printShapeReport(cylinder);
        System.out.println("Material cost: $" +
calculateMaterialCost(cylinder, 2.50));
    }
}
```

Summary & Quick Reference

Inheritance Hierarchy Summary

INHERITANCE CONCEPTS
1. INHERITANCE (extends) • Superclass → Subclass relationship • Code reuse via inherited members • Models "IS-A" relationship 2. CREATING SUBCLASSES • Use extends keyword • Call super() in constructor • Inherit all members (but private not accessible) 3. METHOD OVERRIDING (@Override) • Subclass changes inherited behavior • Same signature as parent method • Use super.method() to call parent version 4. SUBSTITUTION PRINCIPLE • Subclass object → Superclass reference • Cylinder IS-A Circle • Enables polymorphism 5. DYNAMIC BINDING • Method resolved at RUNTIME • Based on actual object type • Enables runtime polymorphism 6. POLYMORPHISM • One interface, many implementations • Same method call, different behavior • Heterogeneous collections possible

Quick Reference Tables

Inheritance Syntax:

```
// Superclass
public class Circle {
    private double radius;
    public Circle(double r) { this.radius = r; }
    public double getArea() { return Math.PI * radius * radius; }
}

// Subclass
public class Cylinder extends Circle {
    private double height;
    public Cylinder(double r, double h) {
        super(r);           // Call parent constructor
        this.height = h;
    }
}
```

```

@Override
public double getArea() { // Override parent method
    return 2 * Math.PI * getRadius() * (getRadius() + height);
}

public double getVolume() { // New method
    return super.getArea() * height; // Call parent's getArea()
}
}

```

Key Concepts Comparison:

Concept	Description
Inheritance	Mechanism to create new class from existing
Subclass	Class that inherits (child/derived)
Superclass	Class being inherited from (parent/base)
extends	Keyword to establish inheritance
super()	Call parent constructor (must be first)
super.method()	Call parent's method version
@Override	Annotation marking method override
IS-A	Relationship type (Cylinder IS-A Circle)
Substitution	Use subclass where superclass expected
Dynamic Binding	Runtime method resolution
Polymorphism	Many forms, one interface

Practice Exercises

Exercise 1: Square Class

Create a Square class that extends Rectangle.

Requirements:

- Constructor should accept one parameter (side length)
- Override toString() to show "Square[side=X]"
- Add a getSide() method
- Test in a Rectangle[] array

Starter Code:

```

public class Square extends Rectangle {
    // Your code here
}

// Test
Rectangle[] shapes = {
    new Rectangle(4, 3),
    new Square(5),
    new Cuboid(2, 3, 4),
    new Square(7)
};

```

```

for (Rectangle r : shapes) {
    System.out.println(r.getClass().getSimpleName() +
        ": Area = " + r.getArea());
}

```

Exercise 2: Person and Student

Create an inheritance hierarchy:

```

public class Person {
    private String name;
    private int age;

    // Constructor, getters, setters
    // toString()
}

public class Student extends Person {
    private String studentId;
    private double gpa;

    // Constructor that calls super()
    // Additional getters, setters
    // Override toString() to include student info
}

```

Test:

```

Person[] people = {
    new Person("Alice", 30),
    new Student("Bob", 20, "S12345", 3.8),
    new Person("Charlie", 25),
    new Student("Diana", 22, "S12346", 3.9)
};

for (Person p : people) {
    System.out.println(p); // Polymorphic toString()!
}

```

Exercise 3: Polymorphic Shapes

Given the Circle and Cylinder classes, write a method:

```

public static Circle findLargestArea(Circle[] shapes) {
    // Return the shape with largest area
    // Use polymorphism - getArea() works for both Circle and Cylinder
}

```

Exercise 4: instanceof Practice

Write a method that:

1. Accepts `Circle[]` (may contain Circles and Cylinders)
2. Returns count of each type
3. Calculates total volume (Cylinders only)

```

public static void analyzeShapes(Circle[] shapes) {
    int circles = 0, cylinders = 0;
    double totalVolume = 0;

    // Your code here

    System.out.println("Circles: " + circles);
    System.out.println("Cylinders: " + cylinders);
    System.out.println("Total cylinder volume: " + totalVolume);
}

```

Exercise 5: Method Overriding

Add these methods to Cylinder and observe polymorphism:

```

public class Cylinder extends Circle {

    // Override getPerimeter() - what should cylinder's perimeter mean?
    // Maybe perimeter of base + perimeter of top?
    @Override
    public double getPerimeter() {
        // Your implementation
    }

    // Override toString()
    @Override
    public String toString() {
        // Your implementation
    }
}

```

Common Mistakes to Avoid

✗ Mistake #1: Forgetting super() in Constructor

```

// ✗ WRONG
public class Cylinder extends Circle {
    private double height;

    public Cylinder(double radius, double height) {
        // Missing super(radius)!
        this.height = height;
    }
}
// Compile error: implicit super() cannot find no-arg constructor in Circle

// ✔ CORRECT
public Cylinder(double radius, double height) {
    super(radius); // Must call parent constructor
    this.height = height;
}

```

✗ Mistake #2: super() Not First Statement

```

// ✗ WRONG

```

```
public Cylinder(double radius, double height) {
    this.height = height;
    super(radius); // Too late! Must be first!
}
```

```
// ✓ CORRECT
public Cylinder(double radius, double height) {
    super(radius); // First statement
    this.height = height;
}
```

✗ Mistake #3: Accessing Private Fields Directly

```
public class Circle {
    private double radius;
}

public class Cylinder extends Circle {
    public void test() {
        // System.out.println(radius); // ✗ ERROR: radius has private
        // access
        System.out.println(getRadius()); // ✓ Use inherited getter
    }
}
```

✗ Mistake #4: Confusing Overriding with Overloading

```
public class Circle {
    public double getArea() { return 0; }
}

public class Cylinder extends Circle {
    // ✗ This is OVERLOADING, not overriding!
    public double getArea(boolean includeBase) {
        return 0;
    }

    // ✓ This is OVERRIDING
    @Override
    public double getArea() {
        return 0;
    }
}
```

✗ Mistake #5: Wrong Direction Casting

```
// ✗ WRONG - Trying to assign superclass to subclass
Circle c = new Circle(5);
Cylinder cy = c; // Compile error!

// ✗ WRONG - Unsafe downcast
Cylinder cy2 = (Cylinder) c; // Compiles but throws ClassCastException!

// ✓ CORRECT - Upcast (automatic)
Cylinder cy = new Cylinder(5, 10);
Circle c = cy; // OK!

// ✓ CORRECT - Safe downcast with instanceof check
```

```
Circle c2 = new Cylinder(5, 10);
if (c2 instanceof Cylinder) {
    Cylinder cy3 = (Cylinder) c2; // Safe!
}
```

✗ Mistake #6: Trying to Call Subclass Methods on Superclass Reference

```
Circle c = new Cylinder(5, 10);

// ✗ Cannot call Cylinder-only methods
c.getVolume(); // Compile error! Circle doesn't have getVolume()
c.getHeight(); // Compile error! Circle doesn't have getHeight()

// ✔ Must downcast first (if safe)
if (c instanceof Cylinder) {
    Cylinder cy = (Cylinder) c;
    cy.getVolume(); // OK now!
}
```

✗ Mistake #7: Forgetting @Override Annotation

```
public class Cylinder extends Circle {
    // ✗ Typo without @Override - creates NEW method!
    public double getAraa() { // Meant to override getArea()
        return 0;
    }

    // ✔ With @Override - compiler catches typo
    @Override
    public double getAraa() { // Compile error: method does not override
        return 0;
    }
}
```

Study Checklist

Before the exam, make sure you can:

- ☐ Explain what inheritance is and why it's useful
 - ☐ Create a subclass using `extends`
 - ☐ Use `super()` correctly in constructors
 - ☐ Override methods with `@Override`
 - ☐ Explain the difference between overriding and overloading
 - ☐ Apply the substitution principle (upcast)
 - ☐ Understand what members are accessible in subclass
 - ☐ Use `super.method()` to call parent's method
 - ☐ Explain dynamic binding and how it works
 - ☐ Demonstrate polymorphism with heterogeneous arrays
 - ☐ Use `instanceof` for safe downcasting
 - ☐ Explain compile-time vs runtime method resolution
-

Next Week Preview

Topics:

- Abstract classes and abstract methods
- The `Object` class and its methods
- Interfaces
- `final` keyword
- Design patterns with inheritance

Preparation:

- Review this week's concepts thoroughly
 - Practice creating inheritance hierarchies
 - Experiment with polymorphic arrays
-

End of Lecture Notes - Week 03

Remember: Inheritance models "IS-A" relationships, enables code reuse, and provides the foundation for polymorphism. Practice is key to mastering these concepts! ☐